# 単一 GPU 上の Static Task Graph 実行における CUDA Stream 配置と SM 資源配分

DAG 構造と実カーネル特性に基づく二段階ヒューリスティック

# 背景：Stream を増やすだけでは速くならない

- 独立な CUDA カーネルは複数 Stream で同時実行できる
- 一方、複数 Stream は同じ GPU の SM を共有する
- 重要な先行タスクの遅延は、後続処理全体へ伝播する
- 起動・同期・Green Context 管理にもコストがある

**理想化した実行時間の下界**

$$T \geq \max\left(\frac{W}{M}, CP\right)$$

$W$: 総仕事量，$M$: SM 数，$CP$: クリティカルパス長

**課題**

並列性だけでなく、依存関係と SM 競合を同時に考える必要がある。

# 研究目的と検証項目

## 目的

Static Task Graph（STG）の DAG 構造と実カーネル特性から、Stream 配置と SM 資源配分を決める方式がどの条件で有効かを明らかにする。

1. 複数 Stream によるタスク並列性は実行時間短縮に寄与するか
2. bottom level による重要度付けと SM 配分は既存方式を改善するか
3. DAG 構造の違いは、提案方式の効果をどう変えるか

## 事前仮説

独立分岐が多い DAG では複数 Stream が効きやすい。長い分岐が合流する DAG では、遅い分岐と合流待ちが性能を支配する。

# 提案手法：二段階ヒューリスティック

**入力**

STG
依存関係 $E$
代表時間 $p_i$
SM 別実測 $g_i(x)$

$\Longrightarrow$

**Stage 1：Stream 配置**

1. bottom level 計算
2. ready タスクを優先
3. 最早終了 Stream へ配置

$\Longrightarrow$

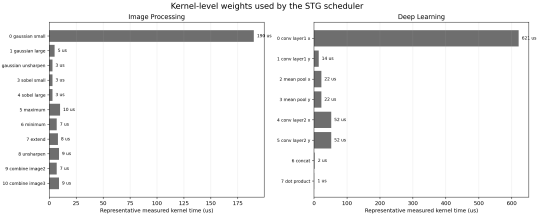
**Stage 2：SM 配分**

1. Stream 配置を固定
2. 8 SM 単位で候補生成
3. 予測 makespan 最小を選択

最後に、決定した構成で実 CUDA カーネルを起動し、CUDA Event で評価する。

# STG は実カーネルと一対一に対応

- タスク ID をランチャ・依存関係・SM プロファイル・出力検証で共有
- `proc_time` は固定条件で測った実カーネルの代表時間
- SM 数 $8, 16, \ldots, 64$ ごとに実測曲線 $g_i(x)$ を作成
- DAG は実行トレースからの自動抽出ではなく、実装のデータ依存から静的に定義




## 3 種類の時間を区別

| 値 | 用途 |
|---|---|
| $p_i$ | bottom level・初期配置 |
| $g_i(x)$ | SM 配分候補の予測 |
| `gpu_kernel_ms` | 最終性能評価 |

# 重要度・Stream 配置・SM 配分の定式化

## bottom level

$$b_i = p_i + \begin{cases} 0 & (S_i = \emptyset) \\ \max_{j \in S_i} b_j & (\text{otherwise}) \end{cases}$$

$b_i$ が大きい ready タスクを優先する。

## Stream 配置

$$r_i = \max_{j \in P_i} f_j$$

$$f(i, s) = e(i, s) + p_i$$

予測終了時刻が最小となる Stream へ配置する。

## SM 配分

$$L_s = \sum_{\sigma(i)=s} \hat{p}_i(x_s)$$

$$\hat{C} = \max_s L_s$$

$\hat{p}_i(x_s) = g_i(x_s)$ を用い、$\hat{C}$ 最小の候補を選ぶ。

## モデルの限界

予測値は、依存待ち・同時実行時の性能変化・起動同期・Context 解放を完全には表さない。

# 関連研究における位置付け

## 直接比較すべき手法

1. **HEFT**：upward rank と挿入型 EFT
2. **PEFT**：後続コストを含む実行先選択
3. **CPOP**：クリティカルパス重視
4. **DLS / HLFET / ETF**：DAG リストスケジューリング
5. **SSGS**：eligible 集合から最早資源可行位置へ配置

## 本手法との主要な差

既存の DAG 方式は主にタスク – プロセッサ割当てを決める。本手法は HEFT 型の配置を単一 GPU 内へ適用した後、配置を固定して Stream 数と Stream 別 SM 数を探索する。

## 比較対象ではなく類推に留まる分野

EDF、WFQ、DRF、DSATUR、勤務表 GA などは、優先度または資源共有の一部だけが類似し、同一問題のベースラインではない。

# 実験条件

## 比較する 4 手法

1. Sequential
2. Existing
3. Existing + Green Context
4. Proposed

## 合成 STG（5 種類）

Fully Parallel、Mixed、Multiple Long Branches、Random DAG、Sequential Chain

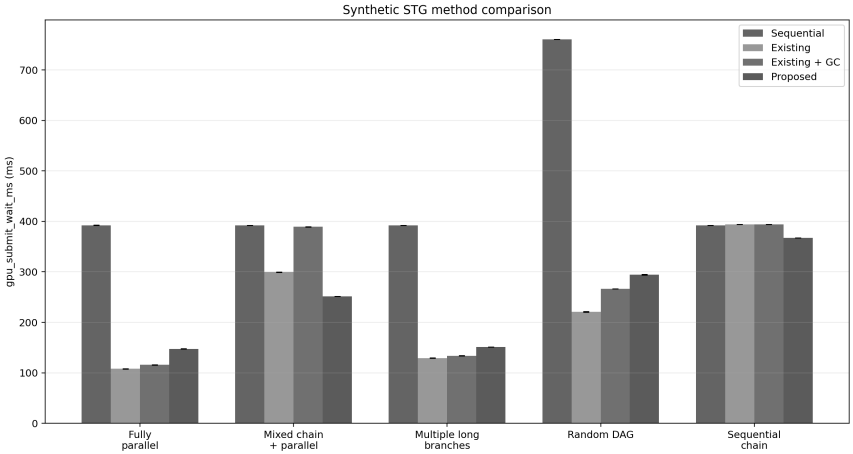
## 実アプリケーション（6 種類）

Vector Square、Black–Scholes、Machine Learning、HITS、Image Processing、Deep Learning

## 測定

- 33 回実行、先頭 3 回をウォームアップとして除外
- 30 サンプルの平均・標準偏差・中央値・95%信頼区間
- 主指標：CUDA Event による `gpu_kernel_ms`
- 実アプリケーションは出力値も検証

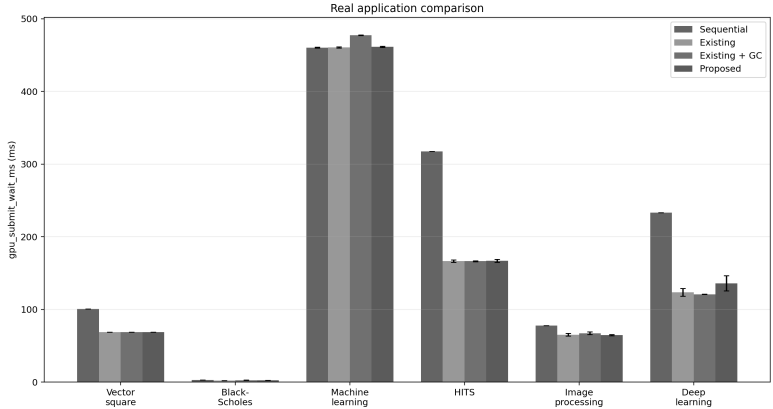
# 結果 1：合成 STG では全 5 種類で Proposed が最短




## 解釈

合成 STG での優位性を、実アプリケーションや異なる GPU へそのまま一般化はできない。

# 結果2：実アプリケーションでは最良手法が異なる

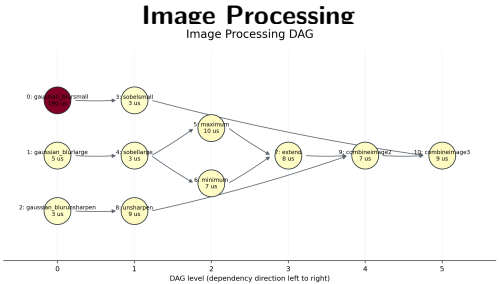



**Proposed が最短：**Image Processing（64.32 ms）

**Existing が最短：**Vector Square、Black–Scholes、Deep Learning

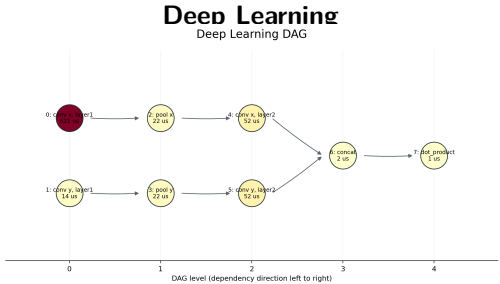
**HITS の Existing と Proposed は数値検証 FAIL のため、性能主張から除外。**

# Image Processing と Deep Learning：DAG 構造の違い




| | |
|---|---|
| 根タスク数 / 最大幅 | 3 / 3 |
| $W$ / $CP$ | 254 / 202 |
| 並列度上限 $W/CP$ | 1.26 |

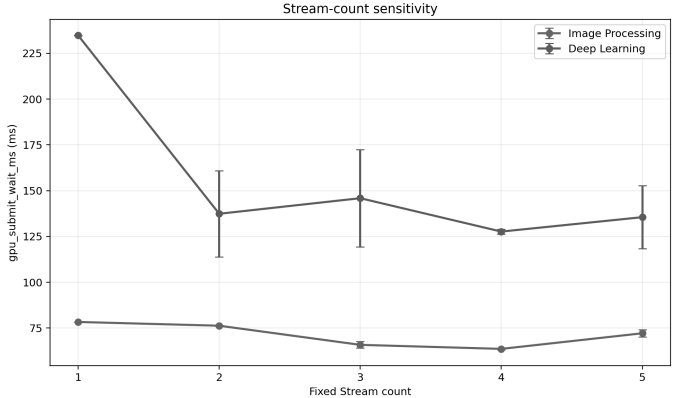
複数の独立分岐を Stream へ分散しやすい。




| | |
|---|---|
| 根タスク数 / 最大幅 | 2 / 2 |
| $W$ / $CP$ | 786 / 698 |
| 並列度上限 $W/CP$ | 1.13 |

長い 2 分岐の遅い方を `concat` が待つ。

# Stream 数感度：多ければよいわけではない




## Image Processing

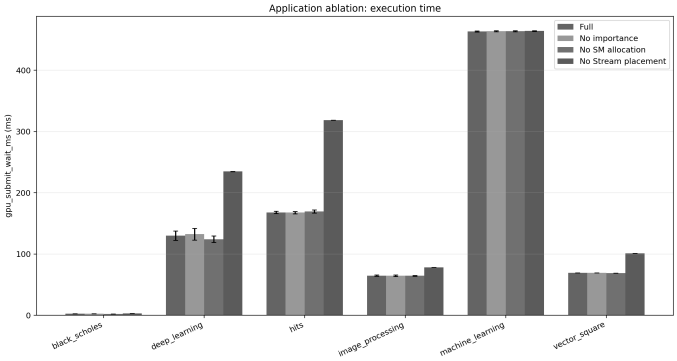
1 Stream: 78.23 ms
**4 Streams: 63.45 ms**
5 Streams: 71.97 ms

## Deep Learning

1 Stream: 234.64 ms
**4 Streams: 127.52 ms**
5 Streams: 135.39 ms

両者とも 4 Stream が最短だが、提案方式全体が既存方式を上回るとは限らない。

# 厳密アブレーション：構成要素の寄与を分離




- Full
- No importance
- No SM allocation
- No Stream placement

**読み方**

比較対象以外の入力・Stream 数・測定方法を統一し、同じ CUDA Event 境界で比較する。

Stream 配置の効果は確認できる一方、重要度と非均等 SM 配分の追加効果は一様でない。

# 正確性と測定区間

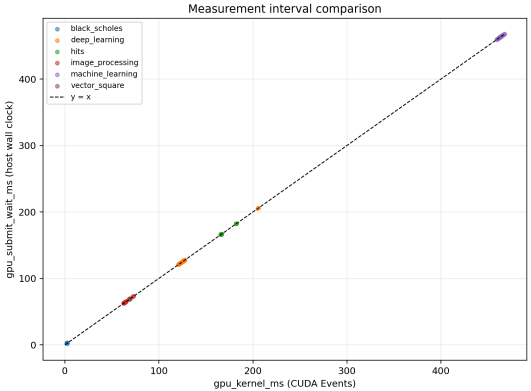
## 数値検証

**22 / 24 条件 PASS**

HITS の Existing・Proposed は FAIL。短い実行時間でも、結果が誤っていれば有効な性能値としない。

## 主指標

`gpu_kernel_ms`：最初のタスク開始から最後のタスク終了までの CUDA Event 時間




`gpu_submit_wait_ms` は投入・待機・資源解放・同期を含むホスト時間であり、主指標と分けて報告する。

# スケジューリングオーバーヘッド

| ベンチマーク | Scheduling | GPU kernel | 比率 |
|---|---|---|---|
| Black–Scholes | 0.139 ms | 2.165 ms | **6.41%** |
| Deep Learning | 0.129 ms | 129.900 ms | 0.10% |
| HITS | 0.210 ms | 167.771 ms | 0.12% |
| Image Processing | 0.145 ms | 64.563 ms | 0.22% |
| Machine Learning | 0.129 ms | 463.279 ms | 0.03% |
| Vector Square | 0.120 ms | 68.920 ms | 0.17% |

- 長時間実行では GPU 時間に対して 0.03–0.22%と小さい
- 短い Black–Scholes では 6.41%となり、無視できない
- 長時間または反復実行される STG で、結果を再利用する設計が適する

# 結論

## 得られたこと

- DAG 重要度に基づく Stream 配置と、実測モデルによる SM 配分を行う二段階ヒューリスティックを実装した
- 合成 STG では 5 種類すべてで Proposed が最短だった
- 実アプリケーションでは Image Processing のみ Proposed が最短で、最良方式は対象ごとに異なった
- 独立分岐が多い DAG では Stream 配置が効きやすく、長い分岐の合流では遅い分岐が支配的になる

## 主張の範囲

提案方式は、DAG 構造とカーネル特性に応じて配置・配分を構成的に決める方式であり、**すべてのグラフで既存方式を上回る方式ではない**。

# 限界と今後の課題

1. グラフ規模評価を全サイズ・全 seed で本計測する
2. Random DAG を複数 seed で集計し、平均と分散を報告する
3. 複数 GPU・異なる入力サイズで SM 実測モデルを検証する
4. 小規模グラフで全探索または整数計画法の最適解と比較する
5. Stream ごとの CUDA Event 区間を測り、予測と実測の差を分析する

## 主張しないこと

「どのグラフでも最速」「SM 使用率最大なら makespan 最小」「GPU や入力に依存せず有効」「厳密最適解を求める」

# まとめ

性能を決めるのは Stream 数だけではない。

**DAG 構造、クリティカルパス、実カーネルの SM 特性**を
一体として扱う必要がある。

# 付録の比較基準：提案手法 $P$

## 問題設定

静的 DAG 上の非プリエンプティブ CUDA カーネルを単一 GPU で実行し、実測または合成の SM– 時間モデルを用いて makespan を短縮する。

1. $BL(i) = p_i + \max_{j \in succ(i)} BL(j)$ により ready タスクを順位付け
2. 既配置区間への挿入を含む EFT 最小の Stream へ配置
3. 1–5 Stream について配置を生成
4. 配置を固定し、合計 114 SM を原則 8 SM 単位で配分探索
5. SM 依存実行時間で DAG と Stream 直列化を再評価し、予測 makespan 最小を選択

## 比較で必ず区別する点

DAG の有無、静的/動的、プリエンプション、目的関数、実行先選択、資源量による処理時間変化、配置後の資源再配分、通信コストの有無。

# CPU・プロセス：HLFET、ETF、MCP

## HLFET（直接類似：高）

**概要：**DAG の後続経路長に基づく静的レベルが高いタスクからプロセッサへ配置する。
***P* との差：**プロセッサ内部の計算資源量を配置後に変更しない。*P* は Stream 配置後に SM 数を探索し、SM 数による処理時間変化も再評価する。

## ETF（直接類似：高）

**概要：**ready なタスク – プロセッサ対の開始・完了時刻を比較し、最早の組を選ぶ。
***P* との差：**ETF は対を一体として選ぶのに対し、*P* は先に bottom level 最大のタスクを固定し、そのタスクについて挿入型 EFT 最小の Stream を選ぶ。

## MCP（直接類似：中 – 高）

**概要：**ALAP 時刻等からクリティカルなタスク順を構成し、依存経路を重視して割り当てる。
***P* との差：**優先度定義が bottom level と同一ではなく、固定能力のプロセッサを前提とする。*P* は明示的なクリティカルパス固定ではなく、全 ready タスクを BL で順位付けする。

# リアルタイム：EDF、LLF、DAG-EDF

## EDF（類推：低）

**概要：**絶対締切が最も早い ready ジョブを優先する。
***P* との差：**優先度は deadline であり DAG の後続経路長ではない。目的は deadline miss 回避で、*P* の単一 DAG makespan 最小化とは異なる。動的・プリエンプティブにも使われる。

## LLF（類推：低 – 中）

**概要：**締切までの余裕から残り処理時間を引いた laxity 最小のジョブを優先する。
***P* との差：**LLF 優先度は時刻とともに変化するが、*P* の BL は実行前に固定される。LLF は deadline を扱い、Stream 選択・SM 配分を含まない。

## DAG-EDF（構造類似：中）

**概要：**先行制約を満たした DAG ノードを EDF 規則で複数プロセッサへ投入する方式群。
***P* との差：**DAG-EDF は DAG ジョブの deadline 保証が中心で、オンライン判断を含み得る。*P* は個別 deadline を持たず、静的配置と SM 構成を makespan で選ぶ。

# 並列・分散計算：HEFT、CPOP、PEFT

## HEFT（最重要の直接類似）

**概要：**平均計算・通信コストによる upward rank 順に、挿入型 EFT 最小の異種プロセッサへ配置する。
*P* **との差：**HEFT は配置時から実行先別時間と通信費を用いる。*P* の Stage 3 は共通 $p_i$ で Stream を選び、配置後に SM 依存時間で Stream 数・SM 数を探索する。*P* は通信コストを明示的に持たない。

## CPOP（直接類似：高）

**概要：**upward/downward rank からクリティカルパスを特定し、パスタスクを選択プロセッサへ集約する。
*P* **との差：***P* はクリティカルパスを抽出・同一 Stream へ固定せず、重要タスクも EFT により分散し得る。CPOP には配置後の SM 分割探索がない。

## PEFT（直接類似：非常に高）

**概要：**Optimistic Cost Table でタスク – プロセッサ別の将来コストを計算し、順位と配置に用いる。
*P* **との差：**PEFT は配置時に実行先別の後続コストを先読みするが、*P* の BL は Stream 非依存である。一方 *P* は配置後に離散 SM 配分を全探索する。

# クラウド・コンテナ：Kubernetes、Firmament、DRF

## Kubernetes Filter–Score（構造類似：低 – 中）

**概要：**資源・制約を満たさない Node を除外し、残る Node を複数規則で採点して Pod を配置する。
***P* との差：**標準方式は Pod 間 DAG やクリティカルパスを直接扱わず、オンラインで 1 Pod ずつ配置する。目的も必ずしも makespan ではなく、配置後の全体 SM 再配分は行わない。

## Firmament（構造類似：中以下）

**概要：**クラスタ配置を min-cost max-flow として表し、監視イベントに応じて全タスクを継続的に再最適化する。
***P* との差：**大規模クラスタの大域的・動的配置であり、BL 順の貪欲構築ではない。目的はポリシー依存の配置コストで、単一 DAG makespan に限定されない。

## DRF（類推：低）

**概要：**CPU・メモリ等についてユーザーの dominant share を均等化する多資源公平配分。
***P* との差：**目的は公平性で DAG を扱わない。*P* は主に単一資源 SM を不均等配分して makespan を下げるため、ユーザー間公平性は目的としない。

# ジョブ・ワークフロー：DLS、Min–Min、方式群

## DLS（直接類似：高）

**概要：**後続経路由来の静的重要度と、実行先別の早期開始効果を dynamic level へ統合し、タスク – プロセッサ対を選ぶ。
*P* **との差：**DLS は対を同時に選び、異種性能・通信費を配置時に扱う。*P* は BL でタスクを先に選び、EFT で Stream を決めた後、SM 量を別段階で探索する。

## Min–Min（構造類似：中）

**概要：**未割当てタスクごとの最小完了時刻を求め、その値が最小のタスクを先に配置する。
*P* **との差：**通常は独立タスクを対象とし短いタスクを優先しやすい。*P* は DAG の後続を含む BL が大きいタスクを優先し、配置後に資源能力も変える。

## Heterogeneous Critical Path Scheduling（名称上の注意）

**概要：**異種環境でクリティカルパスを利用する方式の総称で、単一の標準アルゴリズムではない。
*P* **との差：**数式レベルの比較対象を一意に定められない。論文ではこの総称をベースライン名にせず、HEFT、CPOP、PEFT 等の具体的方式へ置換する。

# ネットワーク：Packet EDF、WFQ、DRR

## Packet EDF（類推：低）

**概要：**deadline が最も近いパケットを先に送信するオンラインキュー制御。
***P* との差：**通常はパケット間 DAG を持たず、目的は期限遵守である。*P* は非プリエンプティブなカーネル DAG 全体の makespan と実行構成を実行前に決める。

## WFQ（類推：低）

**概要：**フローの重みに比例してリンク帯域を公平に共有する。
***P* との差：**公平性・遅延保証が中心で、BL や EFT を用いない。*P* は重要 Stream へ多くの SM を与え得る離散的静的配分であり、公平性を要求しない。

## DRR（類推：低）

**概要：**各キューへ quantum を与え、不足分を次巡へ繰り越す低計算量の公平キューイング。
***P* との差：**巡回型オンライン方式であり、依存グラフや全体 makespan を扱わない。*P* の SM 配分は一度決める固定構成で、実行中に量子を繰り越さない。

# データベース：クリティカル経路、SRPT、Eddy

## Critical-path Query Scheduling（方式群）

**概要：**クエリ DAG 中のクリティカル演算子やパイプラインを優先する考え方の総称。
**$P$ との差：**クエリではデータ量変化、I/O、パイプライン処理が支配的で、固定 $p_i$ の非プリエンプティブカーネルとは異なる。特定論文・方式を指定しない限り正式ベースラインにはできない。

## SRPT（類推：低）

**概要：**残り処理時間が最小のジョブを優先し、平均応答時間を短縮する。
**$P$ との差：**短いジョブを優先するため、長い後続経路を持つタスクを優先する $P$ と逆の選択になり得る。通常は DAG・実行先選択・SM 配分を扱わず、プリエンプティブである。

## Eddy（構造類似：低 – 中）

**概要：**実行時の選択度やキュー状態により、タプルを次の演算子へ動的ルーティングする。
**$P$ との差：**Eddy はタプル単位の適応的実行で、$P$ はカーネル単位の静的配置である。$P$ は事前測定モデルを使い、実行中の観測による再ルーティングをしない。

# 製造・生産：Shifting Bottleneck、CR、LPT

## Shifting Bottleneck Procedure（構造類似：中 – 高）

**概要：**最も重大なボトルネック機械を選び、その機械上の順序を固定し、既決定部分も再最適化する。
*P* **との差：**独立機械の disjunctive graph を反復改善する方式である。*P* は既定 DAG 上で配置を一度構築し、同一 GPU を分割した Stream の SM 量だけを配置後に探索する。

## Critical Ratio（類推：低）

**概要：**納期までの残時間を残り処理時間で割った比により、余裕の小さいジョブを優先する。
*P* **との差：**時刻・納期依存の優先度で DAG 後続長を使わず、実行先・資源量も選ばない。*P* は deadline を入力に持たない。

## LPT（構造類似：中）

**概要：**長い独立ジョブから負荷の小さい固定能力機械へ配置し、makespan を抑える。
*P* **との差：***P* は自身の長さではなく後続経路を含む BL を使う。さらに DAG 先行制約があり、Stream 能力も SM 配分で変化する。

# 人員・勤務表：優先規則、CP、GA

## Priority-rule Scheduling（方式群、構造類似：中）

**概要：**資格、重要度、希望、締切等から作業を順位付けし、順次人員へ割り当てる。
*P* **との差：**一意のアルゴリズムではなく、公平性・労働時間・資格等の多制約が中心である。*P* は BL という固定規則、先行制約、SM 総量、makespan を中心とする。

## Constraint Programming（探索枠組み、類似：低）

**概要：**hard/soft 制約を宣言し、制約伝播と分枝探索で勤務表を求める。
*P* **との差：**CP は汎用探索で最適性を証明できる場合がある。*P* の配置は専用の貪欲法であり、SM 配分だけを限定候補内で全探索するため、全体最適性を保証しない。

## Genetic Algorithm（探索枠組み、類似：低）

**概要：**勤務表を個体とし、交叉・突然変異・選択で多目的評価を反復改善する。
*P* **との差：**GA は確率的で配置自体を繰り返し変更する。*P* は決定的な一方向配置で、固定配置に対する離散 SM 配分のみ網羅する。

# 交通・物流：VRPTW 挿入、ALNS、CR dispatching

## VRPTW Insertion Heuristic（構造類似：中）

**概要：**時間窓・容量を満たしながら、追加距離または終了時刻増分が小さい車両・位置へ配送を挿入する。
*P* **との差：**移動距離、時間窓、車両容量を扱い、DAG 先行制約や SM 数は扱わない。共通するのは既存区間の可行な空き位置へ貪欲挿入する部分である。

## ALNS（改善候補、直接類似：低）

**概要：**解の一部を破壊・再構築し、複数の近傍操作を適応的に選択する。
*P* **との差：**ALNS は配置自体を反復改善し局所最適から脱出するが、*P* は Stage 3 配置を固定したまま Stage 4 へ進む。将来の配置改善法としては利用可能である。

## Critical-ratio Dispatching（方式群、類似：低）

**概要：**納期余裕と残り作業量から遅延危険の高い配送・車両を優先する。
*P* **との差：**DAG の後続経路や実行先能力の探索を含まない。固有の単一物流方式ではないため、正式比較には具体的な問題設定と文献の指定が必要である。

# 教育・施設予約：DSATUR、LDF、Tabu Search

## DSATUR（構造類似：低 – 中）

**概要：**隣接頂点で既使用の色数が最大の頂点を優先するグラフ彩色ヒューリスティック。
***P* との差：**無向競合グラフ上の色数・競合最小化で、処理時間や有向依存を持たない。共通するのは制約が厳しい対象を優先する決定的貪欲構築のみである。

## Largest-Degree-First（類似：低）

**概要：**競合辺が多い授業から時間枠へ割り当てる。
***P* との差：**優先度は無向グラフ次数で、*P* の重み付き有向後続最長路とは異なる。資源能力や処理時間を配置後に変更しない。

## Tabu Search Timetabling（類似：低）

**概要：**授業の移動・交換を行い、Tabu リストで循環を避けつつ制約違反を反復削減する。
***P* との差：**soft constraint を含む局所探索で、構築後も配置を変更する。*P* は単一主要目的の決定的構築で、配置の近傍探索を行わない。

# プロジェクト管理：CPM、SSGS、PSGS

## CPM（分析上の類似：高）

**概要：**DAG の最早・最遅時刻を計算し、余裕ゼロのクリティカルパスを特定する。
*P* **との差：**標準 CPM は資源競合を無視する分析法で、実行先配置を生成しない。*P* は同種の後向き長さを優先度に使い、Stream 競合と SM 容量を含む実行構成を生成する。

## SSGS（構造類似：非常に高）

**概要：**eligible 作業から優先規則で 1 件を選び、先行・資源制約を満たす最早位置へ配置する。
*P* **との差：**SSGS は生成方式であり優先規則は外部指定、通常は作業の資源要求が固定である。*P* は BL を規則として固定し、Stream 順序資源に加えて SM 量で処理時間も変える。

## PSGS（構造類似：高）

**概要：**時間を進め、その時点で ready かつ資源可行な複数作業を並列開始する。
*P* **との差：**PSGS は時間軸を進めるイベント型だが、*P* は 1 タスクずつ将来区間へ挿入する。*P* の Stage 3 は PSGS より SSGS に近く、その後に SM 配分段階を持つ。

# 関連手法から導く本研究の厳密な位置付け

## 位置付け

本手法は、**HEFT 型の DAG リストスケジューリング**を単一 GPU 内の CUDA Stream 配置へ適用し、その配置を固定した後に、**カーネル別 SM スケーリングモデル**を用いて Stream 数と Stream 別 SM 数を探索する二段階静的ヒューリスティックである。

## 独自性を主張するために残る検証

1. 同一 GPU・同一時間モデルで実装した HEFT/PEFT との比較
2. 配置と SM 配分を同時探索する最適解または下界との比較
3. 通信費なし・同種 Stream という条件で、BL+EFT 自体の新規性と SM 探索の寄与を分離
4. 配置固定が最終 SM 配分に対して不利になる反例と性能差の測定

## 直接ベースラインとして推奨

HEFT、PEFT、CPOP、DLS、均等 SM、SM 配分なし、importance なし、配置なし。

# 関連研究の主要文献


- H. Topcuoglu, S. Hariri, and M.-Y. Wu, “Performance-Effective and Low-Complexity Task Scheduling for Heterogeneous Computing,” *IEEE TPDS*, 13(3), 2002. DOI: 10.1109/71.993206.
- H. Arabnejad and J. G. Barbosa, “List Scheduling Algorithm for Heterogeneous Systems by an Optimistic Cost Table,” *IEEE TPDS*, 25(3), 2014. DOI: 10.1109/TPDS.2013.57.
- G. C. Sih and E. A. Lee, “A Compile-Time Scheduling Heuristic for Interconnection-Constrained Heterogeneous Processor Architectures,” *IEEE TPDS*, 4(2), 1993.
- A. Ghodsi et al., “Dominant Resource Fairness: Fair Allocation of Multiple Resource Types,” *NSDI*, 2011.
- I. Gog et al., “Firmament: Fast, Centralized Cluster Scheduling at Scale,” *OSDI*, 2016.
- J. Adams, E. Balas, and D. Zawack, “The Shifting Bottleneck Procedure for Job Shop Scheduling,” *Management Science*, 34(3), 1988.
- R. Kolisch, “Serial and Parallel Resource-Constrained Project Scheduling Methods Revisited,” *European Journal of Operational Research*, 90(2), 1996.


## 注記

“Heterogeneous Critical Path Scheduling”、“Critical-path Query Scheduling”、“Priority-rule Scheduling” 等は単一方式名ではないため、本文で用いる場合は具体的論文へ置き換える。